

SolarBI Pascual

Grafana, gobernanza de datos, automatización ETL e IoT

Consulta de Inteligencia de Negocios

Samuel Alvarez Becerra

Inteligencia de Negocios · Grupo 01 · 2026-II

Docente: Ramiro Grisales Montoya

Institución Universitaria Pascual Bravo

Fecha: 9 de octubre de 2026

Enfoque del grupo: monitoreo operativo (potencia en el tiempo y detección de fallas)

Repositorio: <https://github.com/00Ssam00/SOLARBI-ALVAREZ-BECERRA>

Aclaración. Trabajo individual, autorizado por el docente. Los tableros se construyeron solo en Grafana: el docente autorizó de forma verbal omitir Power BI; el detalle está en la Parte B, Paso 4.

Parte A · Consulta

Máximo 120 palabras por respuesta (sin tablas, diagramas ni código). Al final de cada respuesta se indica el número de las fuentes, listadas en las referencias.

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana?

Grafana es una plataforma de visualización y monitoreo que consulta los datos donde viven, sin copiarlos. Sus piezas son: fuentes de datos (data sources), como el PostgreSQL de SolarBI; paneles, cada uno con una consulta y una visualización; tableros (dashboards), que agrupan paneles; variables, que parametrizan las consultas (por ejemplo, elegir el día); y alertas (alerting), reglas que vigilan una consulta. Grafana OSS es la edición autoadministrada y gratuita, con licencia AGPLv3, y es la que correremos en Docker. Grafana Cloud es el servicio gestionado por Grafana Labs, con capa gratuita para 3 usuarios.

Fuentes: 1

D2. Power BI frente a Grafana

Criterio	Power BI	Grafana
Propósito principal	Análisis e informes de negocio	Monitoreo y visualización operativa
Usuario típico	Analista y directivo	Operador e ingeniero de planta
Conexión a los datos	Importar o DirectQuery; puerta de enlace si la fuente es local	Consulta en vivo a cada fuente de datos, sin copiar
Modelo semántico y lenguaje	Modelo con relaciones y medidas en DAX; transformación en Power Query	Sin modelo; se escribe la consulta del origen (SQL en PostgreSQL)
Actualización y tiempo real	Importado: actualización programada en el servicio, con límite diario según la licencia; DirectQuery: consulta a la fuente en cada interacción	La consulta se ejecuta al abrir o al refrescar el tablero; segundos de latencia
Alertas	Alertas de datos en el servicio (no en Desktop), sobre tarjetas, medidores y KPI	Reglas con varias consultas, estados y puntos de contacto
Licenciamiento y costo	Desktop gratis; Pro USD 14 y Premium por usuario USD 24, por usuario al mes	OSS gratis (AGPLv3); Cloud con capa gratuita; Enterprise por cotización
Control de acceso	Seguridad a nivel de fila (row-level security, RLS) con roles y filtros DAX	Organizaciones, equipos y permisos por carpeta y tablero; RBAC fino solo en Enterprise y Cloud
Versionado y despliegue	Proyecto .pbip en archivos de texto	JSON del tablero y aprovisionamiento por archivos

En SolarBI, Grafana cubre el monitoreo de la potencia; Power BI cubriría el análisis de negocio (energía, ahorro).

Precios consultados en la página oficial el 3 de octubre de 2026; Microsoft aclara que pueden variar por país.

Fuentes: 1, 2, 3, 4, 5, 6, 7, 8, 10

D3. Tableros operativo, analítico y estratégico

Un tablero operativo muestra el estado actual para actuar en minutos (potencia, alarmas). Uno analítico permite explorar causas y tendencias con filtros (rendimiento por día, comparación entre dispositivos). Uno estratégico resume pocos indicadores para decidir (ahorro, energía mensual). En SolarBI, por el enfoque de monitoreo, el tablero principal es operativo y va en Grafana; los indicadores que dependen de tarifas o comparaciones históricas encajan mejor en Power BI por su modelo y sus medidas DAX.

KPI	Tipo	Herramienta	Por qué
Energía diaria (kWh)	Analítico	Grafana	Es una agregación simple sobre fact_energia_dia
Energía total acumulada	Estratégico	Power BI	Medida DAX para comparar periodos y plantas
Yield (kWh/kWp) y Performance Ratio	Analítico	Power BI	El yield usa la capacidad instalada (5 kWp); el PR además la irradiación. PR = yield final / yield de referencia
Ahorro (COP)	Estratégico	Power BI	Combina energía con tarifas, para directivos
Disponibilidad y alarmas	Operativo	Grafana	Exigen alertas y datos casi en tiempo real
% de datos válidos	Operativo	Grafana	Una caída del porcentaje indica fallas de la fuente, junto a la potencia

Fuentes: 20

D4. Grafana con PostgreSQL: series de tiempo y macros

Una consulta de series de tiempo debe devolver una columna llamada `time` (fecha-hora o época Unix en segundos), ordenada por ese campo, con columnas numéricas y el formato "Time series". La macro `$__timeFilter(columna)` se expande a un `BETWEEN` con el rango del selector de tiempo, para traer solo lo visible. La macro `$__timeGroup(columna, '1h')` agrupa el tiempo en intervalos (con TimescaleDB usa `time_bucket`) y `$__timeGroupAlias` además nombra la columna `time`.

```
-- Potencia promedio por hora
SELECT
  $__timeGroupAlias(ts, '1h'),
  AVG(p_ac_kw) AS potencia_promedio_kw
FROM silver.lectura_5min
WHERE $__timeFilter(ts)
      AND dispositivo_id = ${dispositivo} -- variable del tablero
GROUP BY 1
ORDER BY 1;
```

Fuentes: 2

D5. Alertas: Grafana Alerting frente a Power BI

Grafana Alerting evalúa reglas (consultas más una condición) cada cierto intervalo; el estado pasa de Normal a Pendiente y a Alerta cuando se cumple el periodo de espera, y los avisos salen por puntos de contacto (correo, Slack, PagerDuty, webhooks) enrutados con políticas. Las alertas de datos de Power BI solo se crean en el servicio, sobre tarjetas, medidores o KPI numéricos fijados en un tablero, y se evalúan solo cuando el modelo se actualiza; en informes existen alertas de Fabric Activator (correo o Teams), que exigen capacidad Fabric. Para la planta, la regla propuesta es potencia cero a pleno día:

Campo	Valor
Condición	<code>p_ac_kw = 0</code> con <code>irradiancia_wm2 > 100</code>
Ventana horaria	Entre 9:00 y 15:00, hora de Colombia
Umbral	Tres o más lecturas (15 minutos a una lectura cada 5)
Evaluación	Cada minuto; periodo pendiente de 2 minutos en la demostración
Canal	Correo y Slack del equipo de mantenimiento

Fuentes: 3, 8

Bloque 2 · Gobernanza de datos

G1. Gobernanza, gestión y calidad de datos

La gobernanza de datos (data governance) es el ejercicio de autoridad y control (planificar, monitorear, hacer cumplir) sobre la gestión de los activos de datos: define quién decide, con qué reglas y responsabilidades. La gestión de datos (data management) agrupa las funciones que ejecutan esas reglas: arquitectura, almacenamiento, integración y seguridad. La calidad de datos (data quality) es un área de la gestión que mide si el dato sirve para su uso: exactitud, completitud, unicidad. El marco DAMA-DMBOK organiza estas áreas de conocimiento. En SolarBI, la gobernanza decide que la potencia no puede ser negativa, el ETL aplica esa regla (gestión) y el porcentaje de datos válidos mide su cumplimiento (calidad).

Fuentes: 11

G2. Roles de datos

El dueño del dato (data owner) responde por un conjunto de datos y autoriza su uso; el responsable de datos (data steward) define significados y reglas de calidad y vigila su cumplimiento; el custodio (data custodian) opera la infraestructura que guarda y protege el dato. En este trabajo una sola persona cumple los tres papeles, pero se documentan por separado porque el proyecto crecerá con un equipo.

Rol de gobernanza	Rol del equipo SolarBI	Responsabilidad concreta
Dueño del dato	Product owner	Decide qué se mide y quién accede
Responsable de datos	Modelador y analista BI	Define KPI, diccionario y reglas de calidad
Custodio	Ingeniero de datos	Mantiene Postgres, el ETL, respaldos y accesos

Fuentes: 11

G3. Contrato de datos

Un contrato de datos (data contract) es un acuerdo escrito entre quien produce un dato y quien lo consume. Debe incluir esquema, unidades, frecuencia, responsable y niveles de calidad; el estándar ODCS lo organiza en esquema, calidad, acuerdo de servicio y equipo. Si la fuente cambia un campo sin avisar (por ejemplo, p_ac_kw pasa a vatios), el ETL puede fallar al cargar o, peor, aceptar valores 1000 veces mayores: el tablero mostraría picos falsos y la alerta perdería sentido. Por eso se valida el esquema al entrar a Bronze y se rechaza lo que no cumpla el contrato.

Ejemplo simplificado (no sigue el esquema oficial de ODCS; el umbral de 95 % es una propuesta):

```
dispositivo: inversor_01      # frecuencia: 5 minutos, responsable: Samuel Alvarez
campos:
  - {nombre: ts, tipo: timestamp, zona: America/Bogota}
  - {nombre: p_ac_kw, tipo: numerico, unidad: kW, rango: ">= 0"}
calidad: {datos_validos_min: "95 %"}
```

Fuentes: 18

G4. Seguridad a nivel de fila y control de acceso en Grafana

La seguridad a nivel de fila (RLS) de Power BI se define con roles cuyo filtro DAX deja ver solo las filas verdaderas; los miembros se asignan en el servicio y la restricción aplica a quien tenga rol Visor. En Grafana el control equivalente es por organizaciones, equipos y permisos de carpeta o tablero (ver, editar, administrar); los permisos por fuente de datos son de Enterprise y Cloud. No hay RLS nativo: la consulta puede filtrar con \${__user.email}, pero solo protege si el usuario no puede editar el panel. Una barrera sólida son las políticas de fila de PostgreSQL.

```
-- Rol "ResponsableSitio" sobre la tabla dim_dispositivo
[responsable_correo] = USERPRINCIPALNAME()
```

```
-- Equivalente aproximado en una consulta de Grafana
SELECT ... FROM dwh.dim_dispositivo
WHERE responsable_correo = '${__user.email}'
```

Fuentes: 4, 7

G5. Ley 1581 de 2012

Los datos de SolarBI (potencia, irradiancia y temperatura de un inversor) no son personales: la ley define dato personal como información vinculada o asociable a personas naturales determinadas o determinables. Si el tablero mostrara qué operador atendió cada alarma, sí lo serían: habría un titular, un responsable y un encargado del tratamiento, y aplicarían principios como finalidad, libertad (consentimiento), acceso restringido y seguridad. Postura del equipo: recoger lo mínimo, informar la finalidad, restringir el acceso, no publicar nombres ni credenciales y no subir datos personales al repositorio.

Fuentes: 12

Bloque 3 · Automatización de procesos ETL

E1. ETL frente a ELT, capas y Power Query

En ETL (extraer, transformar, cargar) la transformación ocurre antes de cargar, en un servidor intermedio; en ELT se carga el dato crudo y se transforma dentro del destino. SolarBI es ELT: COPY carga el CSV a Bronze y SQL limpia (Silver) y agrega (Gold) dentro de PostgreSQL. Power Query sí es una herramienta ETL: extrae de la fuente, transforma con pasos M y carga al modelo de Power BI; cuando el plegado de consultas (query folding) empuja los pasos a la base, en la práctica se parece a ELT.

Capa	Contenido en SolarBI	Operación
Bronze	data/bronze/telemetry.csv y su tabla cruda	Carga sin modificar
Silver	silver.lectura_5min	Deduplicar, quitar faltantes, validar rango
Gold	dwh.fact_energia_dia	Agregación diaria y energía en kWh

Fuentes: 9, 13

E2. Carga completa, incremental e idempotencia

Una carga completa (full load) reemplaza todo el destino en cada corrida; una incremental procesa solo lo nuevo o cambiado. Una carga es idempotente si ejecutarla varias veces deja el mismo resultado. En fact_energia_dia, la clave primaria (fecha_key, dispositivo_key) impide filas repetidas y INSERT ... ON CONFLICT DO UPDATE actualiza la fila existente en lugar de duplicarla, de forma atómica. Así, correr el ETL dos veces conserva los conteos y reprocesar un día corrige sus valores sin duplicarlos.

```
INSERT INTO dwh.fact_energia_dia (fecha_key, dispositivo_key, energia_kwh, pct_datos_validos)
SELECT fecha_key, dispositivo_key, energia_kwh, pct_datos_validos
FROM tmp_resumen_diario
ON CONFLICT (fecha_key, dispositivo_key)
DO UPDATE SET energia_kwh = EXCLUDED.energia_kwh,
               pct_datos_validos = EXCLUDED.pct_datos_validos;
```

Fuentes: 14

E3. Tres formas de automatizar un flujo

	cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	Planificadores del sistema operativo que ejecutan un comando según una expresión de tiempo; cron usa cinco campos	Plataforma para desarrollar, programar y monitorear flujos definidos como DAG en Python	Función del servicio que recarga un modelo importado; hasta 8 veces al día en Pro y 48 en Premium por usuario
Qué automatiza	Cualquier script, por ejemplo <code>python etl/run_etl.py</code>	Pipelines con varias tareas, dependencias, reintentos y monitoreo	Solo el refresco del modelo, no el ETL
Cuándo conviene	Un solo paso sin dependencias (SolarBI hoy)	Varias fuentes y pasos, con necesidad de visibilidad	Cuando los datos ya están listos en la base

Fuentes: 15

E4. Plegado de consultas

El plegado de consultas es el mecanismo con el que Power Query traduce los pasos de transformación a una sola consulta nativa que ejecuta la fuente (aquí, PostgreSQL), en lugar de traer todo y procesarlo localmente. Mejora el rendimiento porque filtra y agrega donde están los datos y mueve menos filas. Se verifica con "View Native Query", que muestra la solicitud enviada a la fuente cuando el conector lo permite; en Power Query Online hay además indicadores de plegado por paso y un plan de consulta. En SolarBI, la consulta nativa debería incluir el filtro de fechas. Los CSV no se pliegan.

Fuentes: 9

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

Las capas son: dispositivo (inversor con sensores), puerta de enlace (gateway) en el borde que lee el inversor y publica, red o intermediario (broker) que distribuye los mensajes, almacenamiento y aplicación. En el trabajo, `simulador.py` hace de dispositivo y puerta de enlace; el resto de la ruta es real. El diagrama indica la capa de la lectura en cada momento.

```
Inversor -> Gateway -> Broker MQTT -> Archivo CSV -> PostgreSQL -> Grafana
(sensor)   (borde)   (red)      [Bronze]   silver.lectura_5min   panel de potencia
                               simulados   crudo        [Silver] limpio      (lee Silver)
                               [Gold] agregado diario stat de energía
                                                    (lee Gold)
```

Fuentes: 16

I2. MQTT

MQTT es un protocolo de publicación y suscripción (publish/subscribe): los clientes publican mensajes en un tema (topic) y se suscriben a filtros de tema; un intermediario los enruta sin que productor y consumidor se conozcan. QoS 0 entrega como máximo una vez (puede perderse), QoS 1 al menos una vez (puede duplicarse) y QoS 2 exactamente una vez, con más sobrecarga. Para telemetría cada 5 minutos propongo QoS 1: perder una lectura es peor que recibir un duplicado, que el ETL elimina. Los comodines son + (un nivel) y # (varios).

```
pascualbravo/solar/medellin/inversor01/p_ac_kw      (también irradiancia_wm2, temp_modulo_c)
Suscripciones: pascualbravo/solar/+/+/p_ac_kw | pascualbravo/solar/medellin/#
```

Fuentes: 16

I3. Pila abierta de monitoreo IoT

Componente	Papel	Dónde encaja PostgreSQL
Mosquitto	Intermediario MQTT de código abierto (versiones 5.0, 3.1.1 y 3.1)	
Telegraf	Agente con plugins: se suscribe al intermediario y escribe en bases como InfluxDB o PostgreSQL	Telegraf puede escribir directo en PostgreSQL
InfluxDB o TimescaleDB	Base de series de tiempo para almacenar y consultar lecturas; InfluxDB está diseñada para ese fin	TimescaleDB es una extensión de PostgreSQL y Grafana la soporta con <code>time_bucket</code>
Grafana	Visualización y alertas	Lee de cualquiera de las anteriores

En SolarBI, PostgreSQL ya cumple el papel de almacenamiento para Silver y Gold, y podría recibir los mensajes de Telegraf sin cambiar los tableros.

Fuentes: 1, 2, 17

I4. Lotes frente a flujo continuo

Agregar de 5 minutos a un día gana simplicidad, menos almacenamiento y consultas rápidas, pero pierde detalle: una caída de 30 minutos queda oculta en una energía diaria normal y no hay aviso inmediato. El procesamiento por lotes (batch) ejecuta a intervalos, con menor costo y mayor latencia; el flujo continuo (streaming) procesa cada evento con baja latencia y mayor costo. Grafana sobre la tabla de 5 minutos atiende el monitoreo cercano al tiempo real; Power BI, o un panel sobre `fact_energia_dia`, atiende la granularidad diaria.

Fuentes: 13

Bloque 5 · Control de versiones

V1. Git, GitHub, .pbix y .pbip

Git es el sistema de control de versiones distribuido; GitHub es la plataforma que aloja repositorios Git y añade revisión de código, incidencias y solicitudes de cambios. Un `.pbix` es un binario comprimido: Git no puede mostrar qué cambió por dentro ni fusionar versiones. El formato de proyecto `.pbip` guarda el modelo y el informe como archivos de texto en carpetas, aptos para Git, y excluye con `.gitignore` la caché local. Un tablero de Grafana se versiona exportando su JSON y se despliega con aprovisionamiento por archivos.

Término	Definición
Repositorio (repository)	Proyecto completo con el historial de cada archivo
Commit	Instantánea guardada de los cambios preparados con <code>git add</code>
Rama (branch)	Línea de desarrollo separada, para proponer cambios sin tocar la principal
Solicitud de cambios (pull request)	Propuesta de integrar cambios, con discusión y revisión antes de fusionar

Fuentes: 5, 10, 19

Parte B · Flujo ETL y tablero

Se construyó, de extremo a extremo, un flujo de datos con la telemetría simulada de un inversor solar de 5 kWp (una lectura cada 5 minutos durante 3 días) que termina en un tablero de Grafana con una alerta. Cada paso lleva la evidencia capturada en el equipo del autor. Todo corre en Docker y el ETL tiene un único punto de entrada.

CSV crudo → bronze → silver → dwh (hechos) → Grafana → alerta
simulador (texto) (limpio) (energía/día) tablero (regla)

Paso 0 · Infraestructura

Dos contenedores con Docker Compose: PostgreSQL 17 y Grafana 12.4. Los puertos solo escuchan en 127.0.0.1, los datos viven en un volumen nombrado de Docker y las contraseñas salen del archivo .env, que no está en el repositorio.

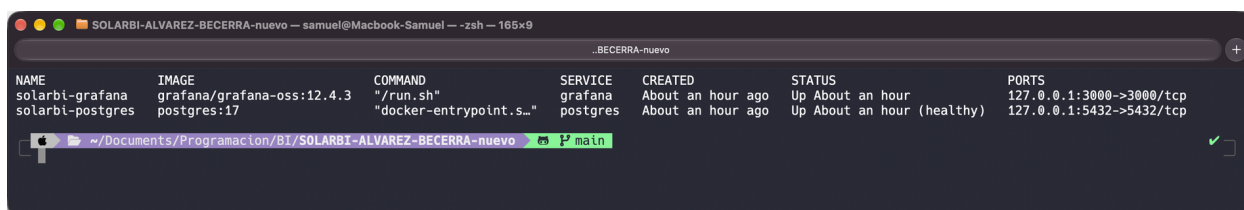


Figura 1. docker compose ps: los dos servicios activos.

Paso 1 · Capa Bronze (dato crudo)

etl/simulador.py genera 887 filas con random.seed(42), así que el archivo es idéntico en cada corrida (hash SHA-256 894eeab5...). Incluye errores de captura (potencia -1, irradiancia vacía y filas repetidas) y una **falla inyectada**: el 2 de octubre de 11:00 a 11:55 la potencia es 0 con sol, para que la alerta tenga algo que detectar. El bronze nunca se modifica.

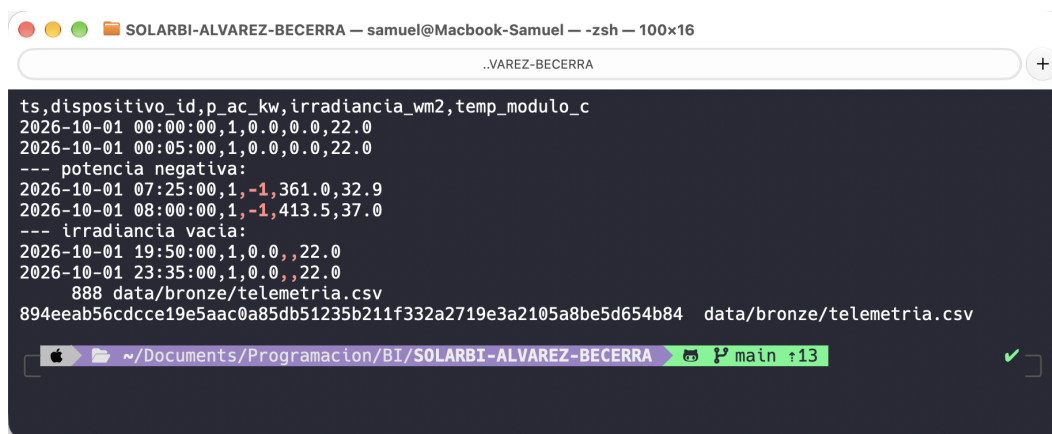


Figura 2. Cabecera, dos filas con potencia -1, dos con irradiancia vacía, 888 líneas y el hash del archivo.

Paso 2 · Capa Silver (reglas de calidad)

La vista silver.v_calidad_filas asigna cada fila a una sola regla, en este orden: duplicado, faltante, formato y fuera de rango ($p_ac_kw < 0$). Por eso las filas leídas son iguales a las válidas más las rechazadas ($887 = 832 + 55$). La potencia 0 es válida: la falla del inversor no se descarta. Las 832 filas válidas (93,80 %) pasan a silver.lectura_5min y se exportan a CSV.

```

SOLARBI-ALVAREZ-BECERRA-nuevo — samuel@Macbook-Samuel — -z...
..BECERRA-nuevo

REPORTE DE CALIDAD (Silver)
concepto                filas    %
filas leídas            887    100.00
rechazadas: duplicado   23     2.59
rechazadas: faltante    15     1.69
rechazadas: formato     0     0.00
rechazadas: fuera_de_rango 17     1.92
filas válidas           832    93.80
comprobación: 887 = 832 + 55 OK

~/Doc/P/B/SOLARBI-ALVAREZ-BECERRA-nuevo main

```

Figura 3. Reporte de calidad: filas leídas, rechazadas por regla, válidas y su porcentaje.

```

SOLARBI-ALVAREZ-BECERRA — samuel@Macbook-Samuel — -zsh — 100x10
..VAREZ-BECERRA

ts,dispositivo_id,p_ac_kw,irradiancia_wm2,temp_modulo_c
2026-10-01 00:00:00,1,0.000,0.0,22.0
2026-10-01 00:05:00,1,0.000,0.0,22.0
2026-10-01 00:10:00,1,0.000,0.0,22.0
833 data/silver/lectura_5min.csv
63eb2e555cbd73fca71b4a05b97fc5a29fda698218313d051d42df7cc19a2e45 data/silver/lectura_5min.csv

~/Documents/Programacion/BI/SOLARBI-ALVAREZ-BECERRA main +13

```

Figura 4. Silver exportado a data/silver/lectura_5min.csv (832 filas más la cabecera).

Paso 3 · Capa Gold e idempotencia

La energía de cada lectura es potencia × 5/60 h. dwh.fact_energia_dia (clave primaria fecha_key, dispositivo_key) guarda por día la energía, las lecturas válidas y el porcentaje de datos válidos (válidas entre 288 esperadas por día, decisión propia). La energía no imputa las lecturas perdidas. INSERT ... ON CONFLICT DO UPDATE hace el proceso idempotente (idempotent): repetir la ejecución no cambia el resultado.

```

SOLARBI-ALVAREZ-BECERRA — samuel@Macbook-Samuel — -zsh — 104x11
..VAREZ-BECERRA

fecha_key | dispositivo_key | energia_kwh | lecturas_validas | lecturas_esperadas | pct_datos_validos
-----|-----|-----|-----|-----|-----
20261001 | 1 | 26.945 | 282 | 288 | 97.92
20261002 | 1 | 22.681 | 274 | 288 | 95.14
20261003 | 1 | 26.306 | 276 | 288 | 95.83
(3 rows)

~/Documents/Programacion/BI/SOLARBI-ALVAREZ-BECERRA main +13

```

Figura 5. Tabla de hechos: energía diaria de 26,945, 22,681 y 26,306 kWh.

```
SOLARBI-ALVAREZ-BECERRA-nuevo — samuel@Macbook-Samuel — zsh — 78x46
..BECERRA-nuevo

ETL SolarBI
1. Esquemas y tablas
  01_esquemas.sql
  02_bronze.sql
2. Bronze
  COPY data/bronze/telemetria.csv -> bronze.telemetria_cruda
3. Silver y Gold
  03_silver.sql
  04_gold.sql

REPORTE DE CALIDAD (Silver)
concepto          filas      %
filas leídas      887      100.00
rechazadas: duplicado    23      2.59
rechazadas: faltante    15      1.69
rechazadas: formato     0      0.00
rechazadas: fuera_de_rango 17      1.92
filas válidas      832      93.80
comprobación: 887 = 832 + 55 OK

CONTEOS (deben ser iguales en cada ejecución)
bronze.telemetria_cruda    887
silver.lectura_5min        832
dwh.dim_fecha              3
dwh.dim_dispositivo        1
dwh.fact_energia_dia       3
suma energia_kwh (fact)    75.932
suma lecturas_validas (fact) 832

RESUMEN DIARIO (dwh.fact_energia_dia)
fecha  dispositivo  energia_kwh  válidas  % válidos
2026-10-01 Inversor 01    26.945     282     97.92
2026-10-02 Inversor 01    22.681     274     95.14
2026-10-03 Inversor 01    26.306     276     95.83

SILVER EXPORTADO: data/silver/lectura_5min.csv (832 filas)

ETL terminado.
```

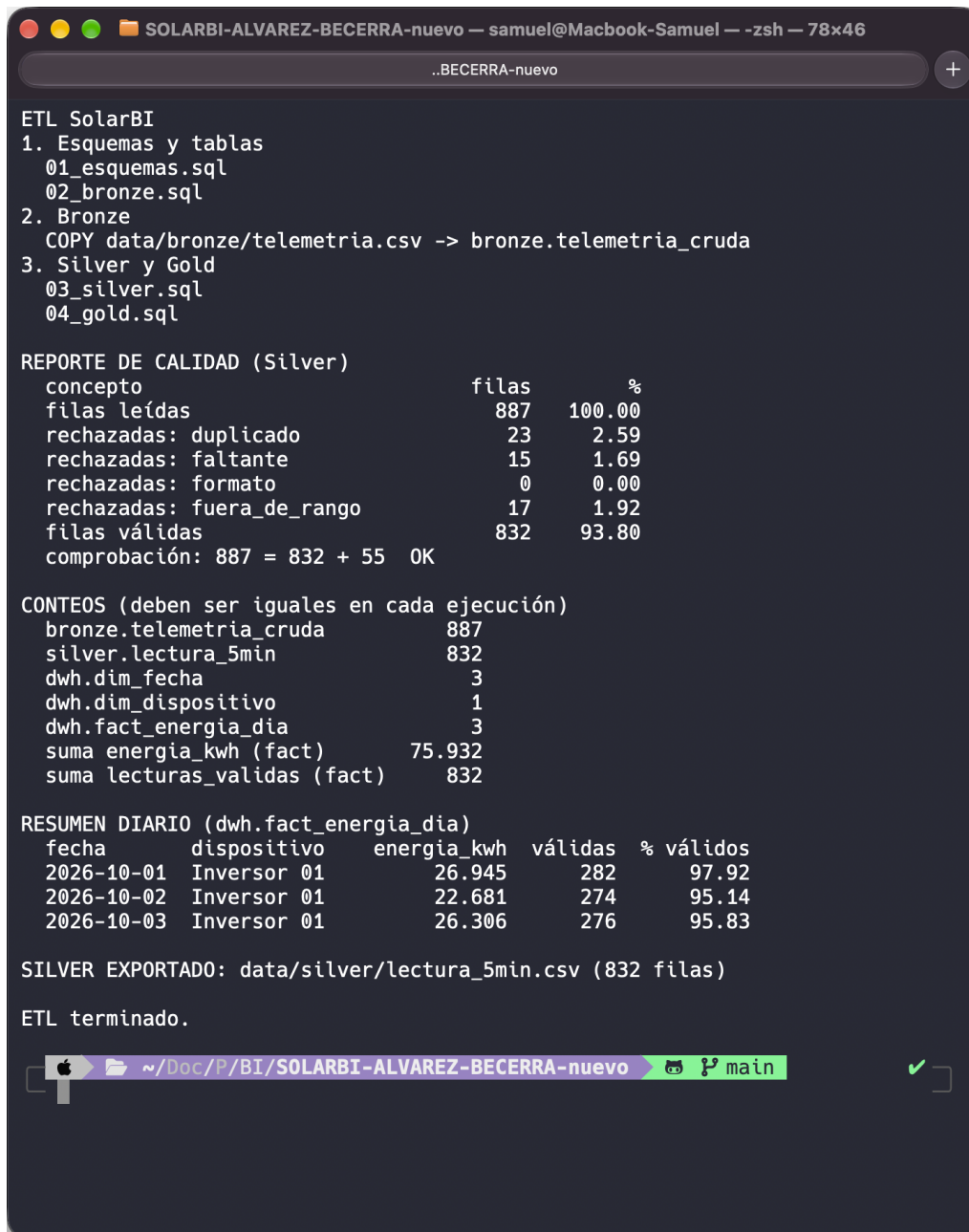
The image shows a terminal window with a dark background. At the top, the window title bar reads "SOLARBI-ALVAREZ-BECERRA-nuevo — samuel@Macbook-Samuel — zsh — 78x46". Below the title bar, the terminal content is as follows: A section titled "ETL SolarBI" with three numbered steps: 1. Esquemas y tablas (with sub-items 01_esquemas.sql and 02_bronze.sql), 2. Bronze (with a COPY command), and 3. Silver y Gold (with sub-items 03_silver.sql and 04_gold.sql). This is followed by a "REPORTE DE CALIDAD (Silver)" section containing a table with 3 columns: concepto, filas, and %. The table lists various data quality metrics for the Silver layer, including total rows read (887), rejected rows (duplicated, missing, format, out of range), and valid rows (832). A summary line shows 887 = 832 + 55 OK. Next is a "CONTEOS" section with a note that values should be equal in each execution, followed by a list of table and view row counts. Then, a "RESUMEN DIARIO" section for the fact table, showing a small table with columns for date, device, energy kWh, valid rows, and percentage valid. The final line of the terminal output is "SILVER EXPORTADO: data/silver/lectura_5min.csv (832 filas)" and "ETL terminado.". At the bottom of the terminal window, there is a status bar showing the current directory as ~/Doc/P/BI/SOLARBI-ALVAREZ-BECERRA-nuevo, the branch as main, and a green checkmark icon.

Figura 6. Un solo comando ejecuta los tres pasos y muestra el reporte, los conteos y el resumen diario.

```

SOLARBI-ALVAREZ-BECERRA-nuevo — samuel@Macbook-Samuel — zsh — 68x...
..BECERRA-nuevo

=== Ejecución 1 ===
CONTEOS (deben ser iguales en cada ejecución)
bronze.telemetry_cruda      887
silver.lectura_5min         832
dwh.dim_fecha               3
dwh.dim_dispositivo         1
dwh.fact_energia_dia        3
suma energia_kwh (fact)     75.932
suma lecturas_validas (fact) 832

=== Ejecución 2 ===
CONTEOS (deben ser iguales en cada ejecución)
bronze.telemetry_cruda      887
silver.lectura_5min         832
dwh.dim_fecha               3
dwh.dim_dispositivo         1
dwh.fact_energia_dia        3
suma energia_kwh (fact)     75.932
suma lecturas_validas (fact) 832

~/Doc/P/B/SOLARBI-ALVAREZ-BECERRA-nuevo main

```

Figura 7. Dos ejecuciones seguidas del ETL: los mismos conteos y la misma energía total (75,932 kWh).

Automatización con cron (solo escrita, no implementada). Ejecuta el ETL todos los días a medianoche; como es idempotente, una repetición no duplica datos.

```

0 0 * * * cd /ruta/al/proyecto && .venv/bin/python etl/run_etl.py >> etl.log 2>&1

schtasks /Create /SC DAILY /ST 00:00 /TN "SolarBI ETL" ^
/TR "cmd /c cd /d C:\ruta\proyecto && .venv\Scripts\python.exe etl\run_etl.py"

```

Paso 4 · Power BI (omitido)

No se construyó. El docente autorizó de forma verbal hacer toda la parte de tableros en Grafana, porque Power BI Desktop solo corre en Windows y este trabajo se desarrolla y se presenta en un solo equipo, un Mac. Cada cifra que pedía Power BI está en el tablero de Grafana, calculada con una consulta SQL sobre el modelo dwh. Los conceptos de Power BI (DAX, seguridad a nivel de fila, plegado de consultas) se responden en la Parte A (D2, G4, E4 y V1).

Lo que pedía Power BI	Dónde está en Grafana
Tarjeta con la energía total (kWh)	Panel «Energía total del rango (kWh)»: 75,93 kWh
Gráfico con la energía diaria	Panel «Energía diaria (kWh)»: barras por fecha, eje desde 0
Tarjeta con el porcentaje de datos válidos	Panel «% de datos válidos del día»: 95,1 % (2 de octubre)

Paso 5 · Tablero y alerta en Grafana

El tablero «SolarBI: monitoreo operativo» (rango fijo del 1 al 3 de octubre, zona America/Bogota, actualización automática apagada) tiene 6 paneles. La serie de potencia usa \$__timeFilter y el promedio por hora usa \$__timeGroupAlias. La fuente de datos, el tablero y la alerta se cargan por archivos (aprovisionamiento, provisioning), sin configurar nada a mano.

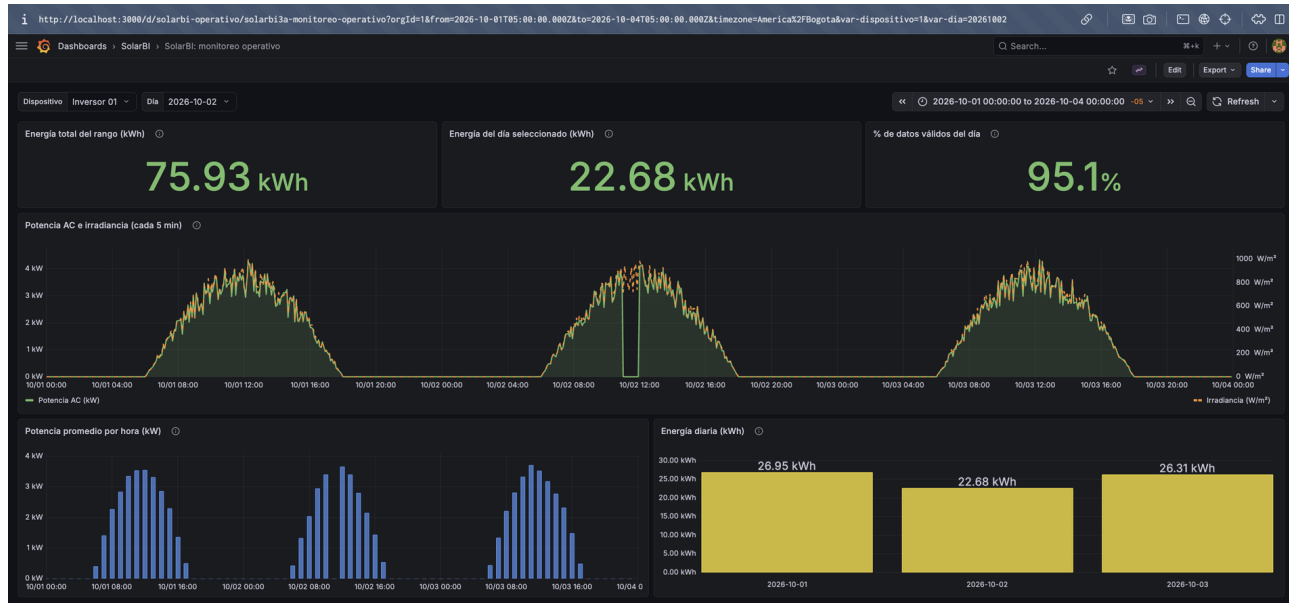


Figura 8. Tablero completo: tarjetas de energía y calidad, potencia, promedio por hora y energía diaria.

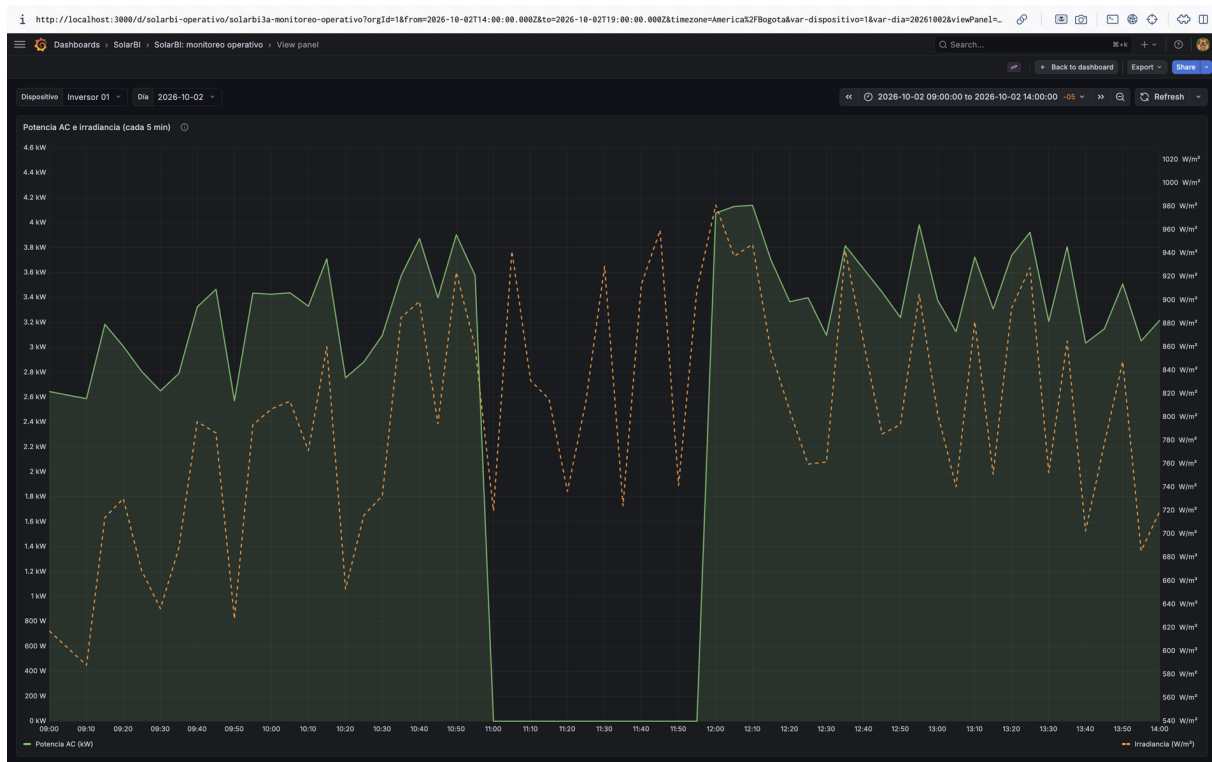


Figura 9. La falla del 2 de octubre: la potencia (verde) cae a 0 de 11:00 a 11:55 mientras la irradiancia (naranja) sigue alta.

Alerta de potencia cero. Cada minuto cuenta las lecturas con $p_{ac_kw} = 0$ e irradiancia mayor a 100 W/m² entre las 9:00 y las 15:00 (hora de Colombia). Con tres o más pasa a pendiente y, tras 2 minutos, a **Firing**. Detecta exactamente las 12 lecturas de la falla. La consulta mira 30 días porque los datos son históricos.

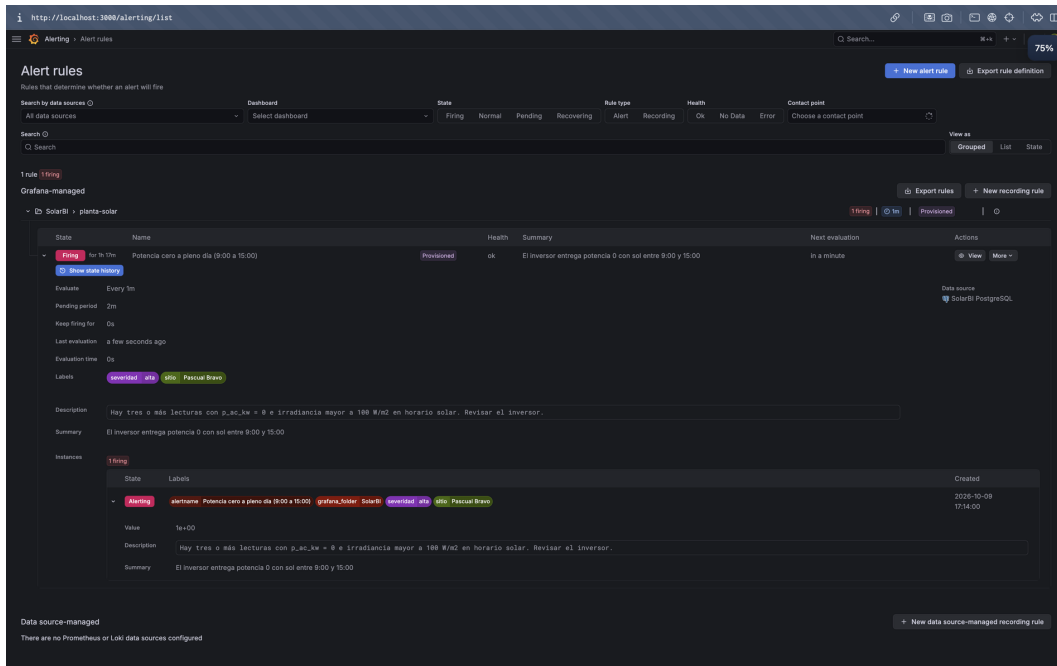


Figura 10. Regla en estado Firing, evaluada cada 1 minuto con periodo pendiente de 2 minutos.

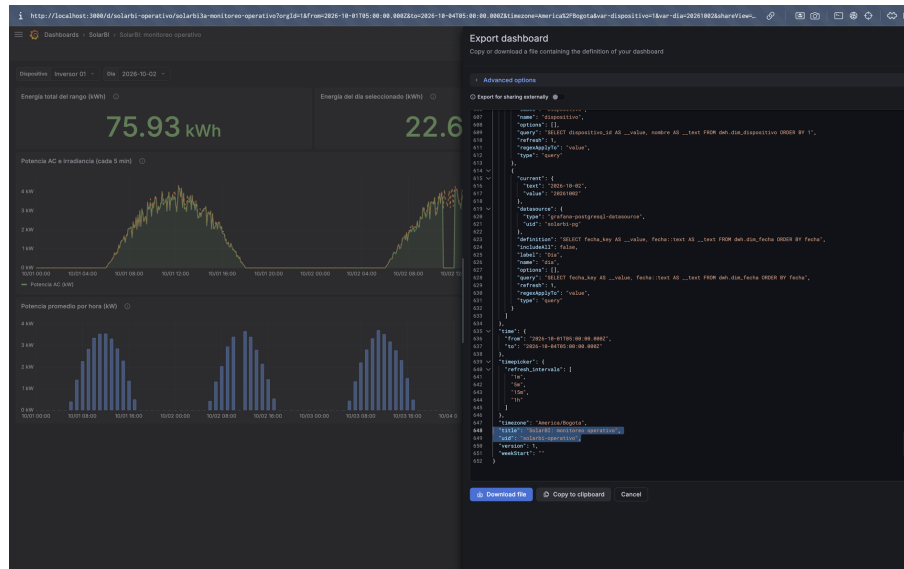


Figura 11. Exportación del tablero como JSON (grafana/dashboard.json), sin credenciales.

Paso 6 · Reflexión

Solo construí la parte de Grafana, así que no puedo decir por experiencia qué resolvió mejor cada herramienta; lo que sigue compara lo que viví con lo que investigué en la Parte A. Grafana me resolvió bien el monitoreo operativo: con una consulta SQL vi la potencia cada 5 minutos, la falla del 2 de octubre se nota a simple vista y la alerta quedó definida por archivos y detecta las 12 lecturas. Lo que más me costó fue acostumbrarme a la forma en que se hacen las cosas en Grafana, por ser un ambiente nuevo para mí. Según lo investigado, el operador necesita ver el estado de la planta y recibir alarmas (Grafana), mientras que el directivo necesita energía, ahorro y comparación entre periodos con un modelo y medidas reutilizables (Power BI, que no construí). Con los más de 4 millones de datos de la segunda etapa tendré más formas de trabajar con ellos y más información útil para el análisis.

Parte C · Repositorio

Repositorio público: <https://github.com/00Ssam00/SOLARBI-ALVAREZ-BECERRA>. Contiene el código, el SQL, la configuración de Grafana, los CSV de bronze y silver, las respuestas de la Parte A y un README con los pasos para reproducir. No incluye credenciales: viven en .env (ignorado por .gitignore) y el repositorio solo trae .env.example, sin valores.

Carpeta	Contenido
data/	bronze/telemetry.csv y silver/lectura_5min.csv
etl/	simulador.py y run_etl.py (único punto de entrada)
sql/	Esquemas, tablas, reglas de calidad, modelo Gold y reporte
grafana/	dashboard.json y provisioning/ (fuente de datos, tablero y alerta)
docs/	parte_a.md
powerbi/	Aviso de omisión y camino para retomar

Cómo reproducir (probado en un clon limpio):

```
cp .env.example .env # completar las dos contraseñas
docker compose up -d # PostgreSQL y Grafana
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python etl/run_etl.py # carga, limpia y agrega
```

Integrante	Aportes
Samuel Alvarez Becerra	Trabajo individual autorizado: simulador y capa Bronze, reglas de Silver, modelo Gold, run_etl.py, Docker Compose, tablero y alerta de Grafana, respuestas de la Parte A, README e informe en PDF

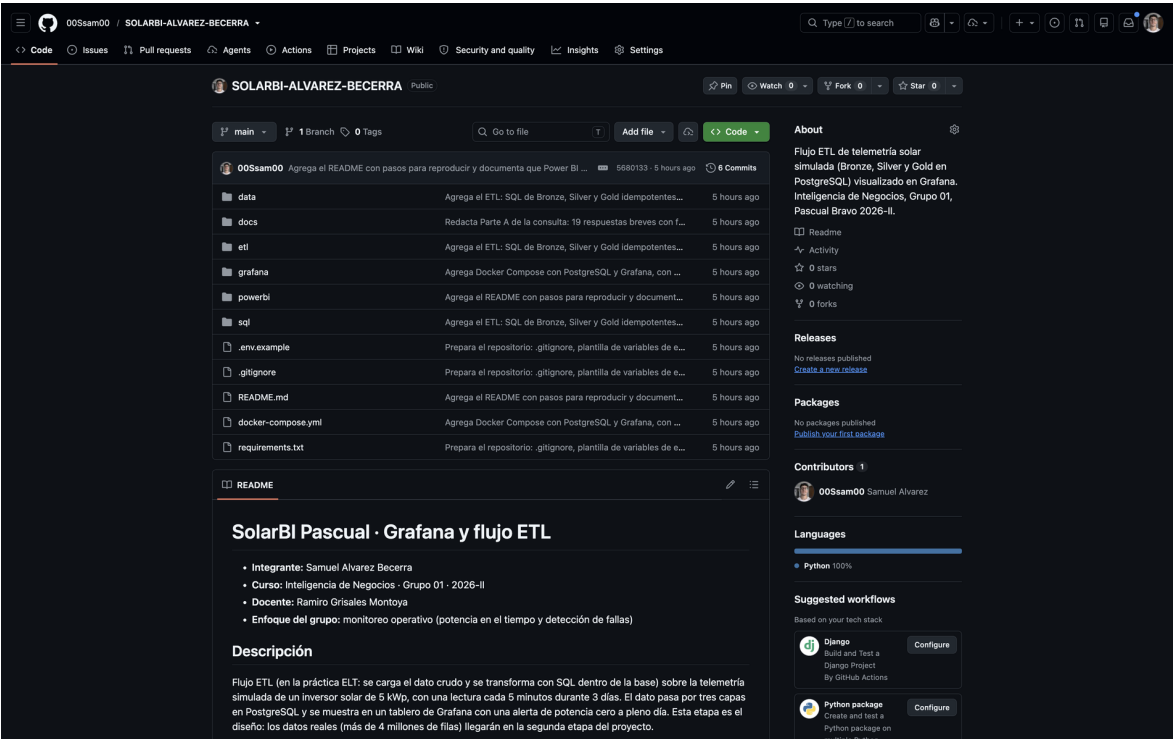


Figura 12. Página del repositorio público: carpetas, archivos y README.

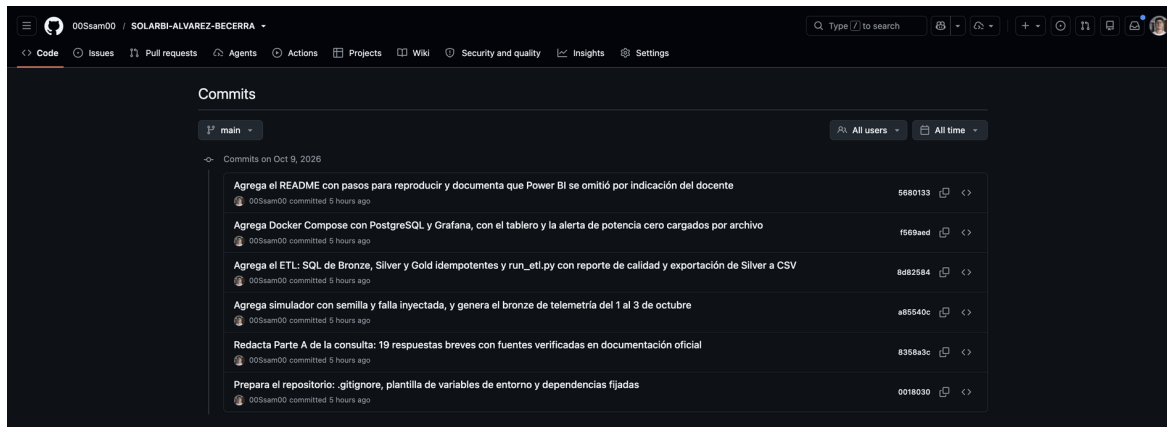


Figura 13. Historial de commits en la rama main, hechos con el usuario 00Ssam00.

Referencias

- [1] Grafana Labs. *Grafana OSS, Cloud y Enterprise*. <https://grafana.com/oss/grafana/>
- [2] Grafana Labs. *Editor de consultas de PostgreSQL (macros y series de tiempo)*. <https://grafana.com/docs/grafana/latest/datasources/postgres/query-editor/>
- [3] Grafana Labs. *Conceptos de Grafana Alerting*. <https://grafana.com/docs/grafana/latest/alerting/fundamentals/>
- [4] Grafana Labs. *Roles y permisos*. <https://grafana.com/docs/grafana/latest/administration/roles-and-permissions/>
- [5] Grafana Labs. *Provisioning*. <https://grafana.com/docs/grafana/latest/administration/provisioning/>
- [6] Microsoft. *Precios de Power BI*. <https://www.microsoft.com/en-us/power-platform/products/power-bi/pricing>
- [7] Microsoft Learn. *Seguridad a nivel de fila (RLS) con Power BI*. <https://learn.microsoft.com/en-us/fabric/security/service-admin-row-level-security>
- [8] Microsoft Learn. *Alertas de datos en el servicio de Power BI*. <https://learn.microsoft.com/en-us/power-bi/create-reports/service-set-data-alerts>
- [9] Microsoft Learn. *Evaluación de consultas y plegado de consultas en Power Query*. <https://learn.microsoft.com/en-us/power-query/query-folding-basics>
- [10] Microsoft Learn. *Proyectos de Power BI Desktop (PBIP)*. <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview>
- [11] DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge*. <https://www.dama.org/cpages/body-of-knowledge>
- [12] Congreso de Colombia. *Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. <https://www.alcaldiabogota.gov.co/sisjur/normas/Norma1.jsp?i=49981>
- [13] Microsoft Learn (Azure Databricks). *Arquitectura medallion (Bronze, Silver, Gold)*. <https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion>
- [14] PostgreSQL Global Development Group. *INSERT (ON CONFLICT)*. <https://www.postgresql.org/docs/current/sql-insert.html>
- [15] Apache Software Foundation. *Apache Airflow*. <https://airflow.apache.org/docs/apache-airflow/stable/index.html>
- [16] OASIS. *MQTT Version 5.0*. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [17] Eclipse Foundation. *Eclipse Mosquitto*. <https://github.com/eclipse-mosquitto/mosquitto>
- [18] Bitol. *Open Data Contract Standard*. <https://bitol-io.github.io/open-data-contract-standard/latest/>
- [19] GitHub Docs. *Acerca de Git*. <https://docs.github.com/en/get-started/using-git/about-git>
- [20] U.S. Department of Energy. *Understanding Solar Photovoltaic System Performance: An Assessment of 75 Federal Photovoltaic Systems* (diciembre de 2021). <https://www.energy.gov/sites/default/files/2022-01/understanding-solar-photovoltaic-system-performance.pdf>